

Chapter 5: Proposed Solution

This chapter presents the system design specifications for the proposed Centralized Digital Management System (CDMS) of Bulgagi Fried Chicken. Building on the operational bottlenecks identified in Chapter 3 and the functional requirements established for the project, this chapter details three core design components: the data design that governs how inventory, sales, and payment information are structured and maintained; the form design that governs how staff and customers interact with the system at the point of transaction; and the report specifications that replace the manual, Excel-based reconciliation process currently used by the business. Interface mockups and screen-level prototypes are developed separately and are presented in the accompanying prototype walkthrough and Appendix; this chapter focuses on the underlying logic, structure, and justification for each design decision.

5.1 Design Principles

The proposed design is guided by three principles drawn directly from the root causes identified in the Ishikawa diagram in Chapter 3.

- Real-time consistency. Every event that affects ingredient stock — a completed sale, a voided order, a spoiled batch — must be reflected in the system within seconds, replacing the current end-of-shift Excel update cycle that leaves the business “blind” to true stock levels during active hours.
- Auditability. Every change to inventory or payment status must be traceable to a specific event, staff member, and timestamp. The business's current weakness is not a lack of digitization (Excel is already used) but a lack of a verifiable trail connecting a stock change to its cause, which is what produces the documented 12% inventory discrepancy rate.
- Error prevention over error correction. Because rush-hour operation is already documented to be chaotic — cashiers are trained to decline orders under pressure — the interface should minimize opportunities for mistakes rather than rely on staff catching them afterward. This follows standard usability heuristics such as error prevention and visibility of system status.

5.2 Data Design

The current setup fails to auto-deduct inventory because the in-store POS and the Excel-based inventory sheet are two disconnected systems, and because Excel itself has no mechanism for expressing that one menu item consumes measured portions of several raw materials (the “Machine” and “Material” root causes in Chapter 3). The proposed data design solves this using a Bill of Materials (BOM) / Recipe structure: every sellable item is decomposed into the exact raw-material quantities it consumes, so that a single POS transaction can automatically trigger the correct deduction across chicken weight, sauce volume, and packaging — regardless of which of the 14 flavor profiles or 3 portion sizes was ordered.

5.2.1 Product Catalog Entities

Menu items are modeled at two levels: the abstract product (e.g., “Boneless Bucket”) and its sellable variant (e.g., Solo / Yangnyeom). This separation exists because a Solo and a Whole bucket of the same flavor consume different raw material quantities and must therefore be linked to the recipe individually, not as a single generic product.

Table	Field	Description
tbl_Products	product_id (PK)	Unique identifier for the base menu item.
	product_name	e.g., “Boneless Bucket,” “Wings Bucket.”
	category	Chicken, Sides, Drinks, Desserts, Rice Meals, etc.
tbl_ProductVariants	variant_id (PK)	Unique identifier for a specific sellable combination.
	product_id (FK)	Links back to the parent product.
	size	Solo / Half / Whole (for buckets).
	flavor	One of the 14 flavor profiles (Yangnyeom, Truffle, etc.).
	sku	Point-of-sale lookup code.
	price	Selling price for this specific variant.

5.2.2 Inventory Ledger Entities

Raw materials are tracked in their smallest usable unit (grams, milliliters, or pieces) so that quantities can never be ambiguous between purchasing units (e.g., a 5kg chicken delivery) and consumption units (e.g., 180g per Solo bucket serving).

Table	Field	Description
tbl_RawMaterials	raw_material_id (PK)	Unique identifier, e.g., “Raw Chicken — Boneless.”
	base_unit	The single canonical unit used for all deductions (g / ml / pcs).
	current_stock_qty	Cached running total, derived from tbl_StockMovements (Sec. 5.2.4).
	reorder_threshold	Minimum safety stock level that triggers the “Out of Stock” state.
	unit_cost	Cost per base unit, used for wastage-cost and COGS reporting.
tbl_Recipe	recipe_id (PK)	Unique identifier for one ingredient line of a variant's BOM.
	variant_id (FK)	Which specific size/flavor combination this line applies to.
	raw_material_id (FK)	Which ingredient is consumed.
	qty_required	Amount of the raw material consumed per unit sold.
	effective_from / effective_to	Date range the recipe was active — see justification below.

5.2.3 Transaction and Payment Entities

A transaction records the order header (channel, staff, status), while a separate line-item table records what was actually sold — this separation is required because a single order (e.g., a Facebook order) typically contains multiple menu items, each of which must independently trigger its own recipe deduction.

Table	Field	Description
tbl_Transactions	transaction_id (PK)	Unique order identifier.
	order_channel	POS (walk-in) / Facebook / GrabFood / Foodpanda.
	status	COMPLETED / VOIDED / REFUNDED / ON_HOLD.
	staff_id (FK)	Cashier who processed the order.
	voided_by / void_reason / voided_at	Populated only if the order is later voided (Sec. 5.2.4).
tbl_TransactionItems	item_id (PK)	Unique line-item identifier.
	transaction_id (FK)	Parent order.
	variant_id (FK)	Specific product variant sold.
	quantity, unit_price, subtotal	Line-item sales detail.
tbl_Payments	payment_id (PK)	Unique payment record.
	transaction_id (FK)	Order being paid for.
	payment_method	CASH / GCASH / QRPH.
	reference_number	GCash/QRPh reference number — UNIQUE across the table (Sec. 5.3.2).
	status	PENDING_VERIFICATION / VERIFIED / REJECTED.
	verified_by (FK)	Staff member who confirmed the payment cleared.

5.2.4 Inventory Movement Ledger

Rather than allowing any process to directly overwrite current_stock_qty, all stock changes — sales, voided-order returns, spoilage, spills, and staff meals — are recorded as individual, non-editable entries in a movement ledger. current_stock_qty is a cached total recalculated from this ledger. This is the mechanism that makes the system auditable: any discrepancy can be traced back to the exact movement, staff member, and reason that caused it, which directly targets the 12% discrepancy problem documented in Chapter 3.

Table	Field	Description
tbl_StockMovements	movement_id (PK)	Unique identifier for one stock-affecting event.
	raw_material_id (FK)	Which ingredient was affected.
	movement_type	SALE_DEDUCTION / VOID_RETURN / RESTOCK / WASTAGE / STAFF_MEAL / SPILLAGE / MANUAL_ADJUSTMENT.
	qty_change	Negative for deductions, positive for returns or restocks.

Table	Field	Description
	reference_transaction_id (FK, nullable)	Links back to the sale/void that caused this, if applicable.
	reason_code (FK, nullable)	Controlled list for non-sale adjustments (see tbl_ReasonCodes).
	performed_by / approved_by (FK)	Staff who logged it, and manager who approved it (required for wastage).
	created_at	Timestamp of the movement.
tbl_ReasonCodes	reason_code_id (PK), label	Controlled vocabulary, e.g., “Spoiled – Held Too Long,” “Dropped/Spilled,” “Staff Meal.”

5.2.5 Design Justification

Variant-level, versioned recipes. Linking recipes to the specific size/flavor variant, rather than the abstract product, ensures a Solo bucket and a Whole bucket of the same flavor deduct different chicken and sauce quantities. Recipes additionally carry effective_from / effective_to dates rather than being overwritten in place, so that if portion sizes are adjusted in the future, historical sales continue to resolve against the recipe that was active at the time of sale — preserving the accuracy of past cost and inventory reports.

Movement ledger instead of a single mutable stock number. This directly resolves the requirement to handle voided orders and non-sale stock changes (spoilage, spills, staff meals) without corrupting sales logs. A void does not delete the original transaction — it flags the transaction's status as VOIDED and generates a new VOID_RETURN movement for each ingredient in that order's recipe. Sales reports simply filter for COMPLETED status, so revenue is never inflated by voided orders, while the full history of what happened (including who voided it and why) remains intact for audit purposes. The same ledger absorbs spoilage and staff-meal adjustments through dedicated movement types, with manager approval required for high-value write-offs — addressing the “Manpower” root cause of unrecorded shrinkage.

Concurrency handling for simultaneous orders. Because the system must sync inventory across the POS, Facebook, GrabFood, and Foodpanda within seconds, two customers on different channels could attempt to purchase the last available portion of an ingredient at nearly the same moment. Stock deductions are performed as a single atomic conditional update (UPDATE ... SET current_stock_qty = current_stock_qty - required_qty WHERE current_stock_qty >= required_qty), so the database itself guarantees only one of two simultaneous requests can succeed; the losing request is rejected before the kitchen is notified. For online-channel orders specifically, where a few minutes may pass between order placement and GCash/QRPh confirmation, the required raw materials are placed in a short-lived reservation (tbl_StockReservations, with an expiry) rather than deducted immediately — this prevents an unconfirmed online order from silently blocking a walk-in sale, while still holding the stock long enough for the payment to be verified.

5.3 Form Design

Form design targets the two operational moments where the current process breaks down: the point of sale, where stock availability must be checked instantly rather than visually, and the payment step, where manual GCash/QRPh verification currently adds 5 to 7 minutes per Facebook order and contributes to the 8-to-10-minute rush-hour delay documented in Chapter 3.

5.3.1 In-Store POS Order Form

- Each product tile queries live ingredient availability against `reorder_threshold`. Two states are used rather than a single binary flag: an amber “Low Stock” badge when an ingredient is within a configurable buffer of its threshold, and a greyed-out “Out of Stock” badge once the threshold is reached — giving cashiers a chance to proactively offer alternatives before a customer commits to an unavailable item.
- The “Confirm Order” button is disabled immediately upon first tap and re-enabled only if the transaction fails server-side, preventing accidental double-submission (and duplicate stock deduction) from a stressed cashier double-tapping during a rush.
- Voiding a completed order or applying a discount above a configured percentage requires a manager PIN entered on the same screen, rather than being a single cashier-level action — this both prevents accidental voids and creates the `approved_by` trail described in Section 5.2.4.

5.3.2 Online / Facebook Checkout Form (GCash & QRPh)

The current process already requires a cashier to manually open the business's GCash account and cross-check that a payment cleared — this is what produces the 5-to-7-minute delay per order. The proposed form does not eliminate manual confirmation outright (full payment-gateway integration is outside the current project scope), but it removes the opportunities for error and fraud within that manual step, and structures the data so the verification step itself is faster:

Field	Rule	Purpose
Payment Method	Radio selection: GCash / QRPh / Cash	Determines which fields render next.
Reference Number	Required, exactly 13 numeric digits, numeric keypad input	Matches the format GCash/QRPh actually issues; numeric keypad avoids mistaps versus a generic text field on mobile.
Reference Number – uniqueness	Rejected at the database level if it already exists in <code>tbl_Payments</code> (UNIQUE constraint)	Prevents a reused or previously-submitted reference number from being accepted twice — a 13-digit format check alone cannot catch a duplicate or fabricated number.
Payment Screenshot	Required image upload attached to the order	Gives the cashier a visual artifact to check against the business GCash app during

Field	Rule	Purpose
		verification, rather than trusting the typed number alone.
Order Status	Set to ON_HOLD (kitchen not notified) until a staff member marks the payment VERIFIED	Mirrors the existing BPMN safeguard against cooking food before payment clears, but now logged with a timestamp and a verifying staff ID instead of being tracked informally.
Submit Button	Disabled until all required fields are valid; disabled again immediately after first tap	Prevents incomplete submissions and duplicate order creation.

It is worth noting for the design defense that a 13-digit format check only confirms that the customer typed something that looks like a reference number — it cannot confirm the payment actually happened. The uniqueness constraint and screenshot requirement above are the two safeguards within the current project scope that reduce this risk; a longer-term improvement, noted as a recommendation for future iterations, would be integrating a Philippine payment gateway aggregator (e.g., PayMongo or Xendit) so that GCash confirms payment directly to the system via webhook, removing manual verification entirely.

5.3.3 Manual Stock Adjustment Form

A dedicated form — separate from the sales flow — lets staff log spoilage, spills, or staff meals without those quantities ever being recorded as a sale. The form requires a raw material, a quantity, and a reason selected from the controlled tbl_ReasonCodes list (rather than free text, so the data stays analyzable), and routes any adjustment above a configured value threshold to a manager-PIN approval step before it is committed — addressing the requirement for a review layer over shrinkage.

5.3.4 Error-Prevention Principles Applied

- Visibility of system status: live stock badges and a persistent “orders in queue” counter keep cashiers aware of state without needing to ask the kitchen.
- Constraints over instructions: the interface makes invalid actions impossible to submit (e.g., a malformed reference number, a sale of an out-of-stock item) rather than relying on a cashier to notice and self-correct under pressure.
- Confirmation for destructive actions: voids, discounts, and high-value stock write-offs require a second step (manager PIN) precisely because they are the actions with the highest cost if triggered by mistake or misused.
- Recognition over recall: menu items are grouped visually by category with flavor/size selectors shown as tap targets rather than typed input, reducing the cognitive load of tracking 14 flavor profiles across 3 portion sizes during a rush.

5.4 Report Specifications

The reports below replace the manual, end-of-shift Excel reconciliation process described in Chapter 2, and are generated automatically from the ledger and transaction data described in Section 5.2 — no report requires a staff member to manually re-total figures from separate sources.

Report	Frequency	Key Fields	Problem Addressed
End-of-Shift Reconciliation Report	Auto-generated at shift close; on demand	Total sales by channel, cash counted vs. system total, GCash/QRPh payments by status (Verified / Pending / Rejected), variance flag	Replaces the manual cross-checking of cash, online payments, and POS reports that currently consumes staff time at shift end.
Inventory & Stock Movement Report	Real-time, with an end-of-shift summary view	Current stock per raw material, movements by type (sales, voids, wastage, staff meals, restocks), items nearing reorder threshold	Directly targets the 12% average inventory discrepancy between Excel logs and physical counts by making every deduction traceable.
Stock-Out Incident Report	Daily / weekly rollup	Raw material, date/time crossed zero, duration out of stock, menu items disabled as a result, estimated missed orders	Tracks the documented 4–5 monthly stock-out incidents so management can measure whether the system is reducing this frequency over time.
Payment Verification Report	Daily / weekly rollup	Reference numbers submitted, verification turnaround time, rejected/duplicate reference number attempts	Measures whether the 5–7 minute per-order GCash/QRPh verification delay is actually decreasing, and surfaces suspicious duplicate reference number attempts for review.
Demand Forecast Report	Weekly, with holiday/peak-period overrides	Historical order volume by product variant, projected raw chicken and sauce requirements for the upcoming shift/week	Supports the predictive forecasting capability identified as a management priority, reducing both raw ingredient waste and unnecessary early menu-item disabling.

Each report is filterable by date range and sales channel, and the End-of-Shift, Inventory, and Payment Verification reports specifically are designed to be reviewable within the same interface cashiers already use, so that the reconciliation step that currently happens in a separate Excel file becomes a byproduct of normal system operation rather than an additional task performed at the end of a shift.